

AI Augmented Learning

Minor AI for Society

Casey Stijlaart

Table of Contents

Introduction.....	1
Used Tools.....	1
Examples.....	2
ChatGPT Codex : Code Review.....	2
ChatGPT Codex : HTML and CSS Snippets	3
ChatGPT 5.3 : Brainstorming and Ideation.....	4
ChatGPT 5.3 : Brainstorming and Ideation - 2.....	5
ChatGPT 5.3 : Documentation Review.....	6
Schematik.io : Hardware Design	8
Claude by Anthropic : Brainstorming, ideation, and Image Generation	10
Claude Code by Anthropic : Code Review and Debugging, Image Generation	12

Introduction

During the semester there is made use of learning outcomes, which must be achieved by the end of the semester. To achieve these learning outcomes, various tools and resources are utilized, including AI technologies. This document outlines the AI tools used during the semester and provides examples of their application in different contexts.

The learning outcome itself is defined as follows:

You use conversational AI to learn, plan, critique, and reflect. You set boundaries, cite AI contributions, and demonstrate growth over time.

— Learning Outcome AI Augmented Learning

When it comes to conversational AI, I have utilized various tools. Each tool has its own strengths and weaknesses, and I have used them in different contexts to achieve specific goals. To outline the tools I have used, I will provide examples of how I have applied them in different scenarios, such as code review, brainstorming, documentation review, and hardware design.

Used Tools

- ChatGPT Codex
 - Used for code review and debugging.
 - Used for small HTML and CSS snippets to deploy a web app.
- ChatGPT 5.3
 - Used for brainstorming and ideation.
 - Used for reviewing documentation and providing feedback.
- Schematik.io
 - Used for creating and visualizing hardware schematics and circuit designs.
- Claude by Anthropic
 - Used for brainstorming, verification, and critique of ideas and concepts.
 - Image related tasks, such as generating images for the startup screen of the app and creating a logo for the project, as it was able to create such things much better than ChatGPT Codex, which struggled with image related tasks and often produced subpar results.
- Claude Code by Anthropic
 - Used for code review and debugging, similar to ChatGPT Codex, but with a different approach and capabilities.
 - Main usage was for an flutter front-end IOS app, as it provided better support for and understanding when it came to my vision for the app.

Examples

For each of the tools mentioned above, I will provide examples of how I have used them in different contexts, along with the outcomes and reflections on their effectiveness.

ChatGPT Codex : Code Review

A good example of using ChatGPT Codex for code review is when I was working on the software development aspect of my personal project, a plant health monitoring system. I had written a piece of code that was supposed to read data from a sensor and process it to determine the health status of the plant. However, I started to notice the program started to take up a significant amount of memory, which could lead to problems in the long run, especially if the program was running for an extended period of time.

"regarding to sem-7-ai/personal-project/src/prototype, I am working on an esp32 cp210x, now i would like some feedback and suggestion regarding to program and flash size. Currently these are the results:

```
text    data    bss     dec     hex filename
915252  228052   25457  1168761  11d579 .pio\build\esp32_com3\firmware.elf
RAM:    [==          ]  15.8% (used 51632 bytes from 327680 bytes)
Flash:  [=====  ]  85.5% (used 1121021 bytes from 1310720 bytes)."
```

— ChatGPT Codex Prompt

I decided to use ChatGPT Codex to review my code and help me identify any potential issues, suggest improvements, and optimize the code for better performance. I provided ChatGPT Codex with the entire project and asked it to review the code and provide feedback on how to optimize it for better performance and memory usage. The response I got was quite helpful, as it pointed out specific areas of the code that were causing the memory issues and provided suggestions on how to optimize it. For example, moving UI assets out of C++ string concatenation, compress/quantize model export, and move away from local storage and local webpage hosting. While this was a good starting point, I had to use my own knowledge and judgment to implement the suggested changes and ensure that they were appropriate for my specific use case and constraints.

For the full code review and suggestions, please refer to the following link: [ChatGPT Codex Code Review](#).

ChatGPT Codex : HTML and CSS Snippets

Another example of using ChatGPT Codex is when I was working on the web development aspect of my project, specifically creating a local web app for the initial proof of concepts and prototype to display the plant health data. It did not need to be a complex web app, but it had to be simple and responsive, as I wanted to display the data in a clear way that was easy to understand and to navigate. As I had near zero experience with web development, I decided to use ChatGPT Codex to help me generate small HTML and CSS snippets that I could use to quickly create the web app. This allowed me to save time and focus on the functionality of the app rather than spending a lot of time on understanding and learning HTML and CSS from scratch. As I was having a chat within ChatGPT Codex already regarding to a similar topic on the same github branch, I decided to ask it for help with generating the javascript, HTML, CSS snippets for the web app in the same conversation, as I thought it would be more efficient to have everything in one place and to be able to easily refer back to it if needed. The response I got was quite helpful, as it provided me with the necessary code snippets to create a simple and responsive web app that met my requirements, see figure [Web App Code Snippets from ChatGPT Codex](#) for an example of the generated code snippets and conversation.

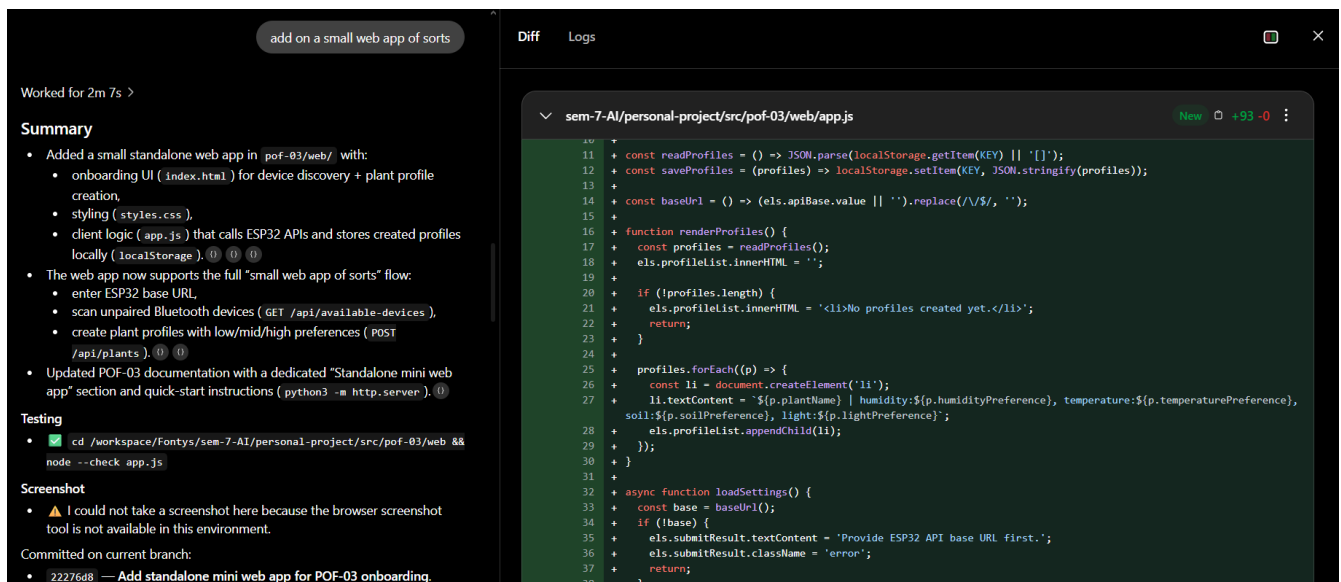


Figure 1. Web App Code Snippets from ChatGPT Codex

Throughout the process of the personal project, I continued to adapt the web page for added functionality, thus I continued to ask ChatGPT Codex for help with generating additional code snippets and making adjustments to the existing code as needed.

This is a clear example of how AI can be used to augment human learning and problem-solving, especially for those who may not have a lot of experience in a specific area, such as web development in this case. By providing AI-generated code snippets, it can help individuals quickly create functional applications without needing to spend a lot of time learning the underlying technologies, while still allowing them to focus on the specific functionality and design of their application.

ChatGPT 5.3 : Brainstorming and Ideation

A good example of using ChatGPT 5.3 for brainstorming and ideation is when I was trying to come up with a more tangible idea on how to make my personal project, a plant health monitoring system, more interactive and engaging for users. I asked ChatGPT 5.3 for suggestions on how to make the system more interactive, and it provided me with several ideas. While some suggestions were great, there were also some that weren't as good. This as it wasn't particularly great for a fragile indoor plant, and next to that it initially suggested many screen bound solutions, which I had already ruled out as I wanted to create a more physical and tangible product. However, it did lead to a sparring sessions where I could bounce ideas off of the AI and refine my concept further.

The initial prompt I used was:

"What are fun and interactive ways to have a plants health be more immersive? It should consider what is not directly harmful to the plant. Example scenerio's are when the plant has perfect health, moderate stress or high stress."

— ChatGPT 5.3 Prompt

And a part of the response I got was:

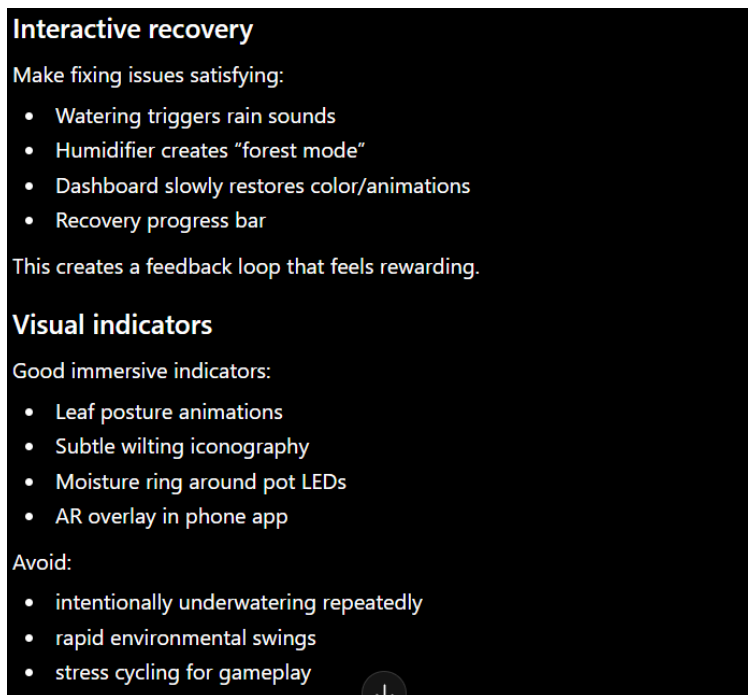


Figure 2. Example Response from ChatGPT 5.3

For the given example of how ChatGPT 5.3 was used only a part of the response is shown, as it was quite long and contained many suggestions. Beside the multiple suggestions, it also included a sparring session of where we went back and forth on the ideas, which is not shown in the figure. The entire conversation was quite lengthy, but it provided me with a lot of inspiration and ideas to work with, which I could then refine and adapt to fit my specific needs and constraints for the project. For the full conversation, please refer to the following link: [ChatGPT 5.3 Brainstorming Session](#).

ChatGPT 5.3 : Brainstorming and Ideation - 2

Another example of using ChatGPT 5.3 for brainstorming and ideation is when I was trying to come up with ideas and feasible solution for allowing for non-local access to the plant health monitoring system, as I wanted to be able to access the data and control the system remotely, without needing to be physically present near the plant. I asked ChatGPT 5.3 for suggestions on how to achieve this, yet it initially struggled to provide feasible solutions that met my requirements and constraints, as it suggested many solutions that were either too complex, too expensive, or not suitable for my specific use case. However, through a back-and-forth conversation and sparring session, it eventually provided more viable options, which I could then evaluate and adapt to fit my specific needs and constraints for the project.

"All right, so I'm working on my plant health monitoring system, and the general prototype is there, but it's all local through a web app as well. Now, the problem is that the current flash is at 85%, so for long term it's definitely not great, especially as all the data is stored locally on the ESP32. Now, what I was thinking, either I need to run MQTT and then, or something, and then have it integrated into a dashboard, like an existing platform such maybe Grafana, or I could integrate MongoDB on the ESP32 and then use it within like Grafana, for example. But I'm not really sure or entirely sure what that would mean for, um, example, program size, because the flash and the program is just getting out of hand, especially because I have a lot of stuff on there. So I need some advice on what would be the best approach, but also something that is not meaning that I need to throw my whole project around, like a 180. i can run something on my laptop if necessary, but i prefer not to, and it would be great if my data isn't just stored for a month. i would like to keep it until i decide to delete it. e.g. influxdb only allows for 30 days storage."

— ChatGPT 5.3 Prompt

For the full conversation, please refer to the following link: [ChatGPT 5.3 Brainstorming Session Non-Local Solutions](#).

ChatGPT 5.3 : Documentation Review

As I personally struggle with documentation and writing (grammar) in general, I found it helpful to use ChatGPT 5.3 for reviewing my documentation and providing feedback on how to improve it. I would provide ChatGPT 5.3 with a draft of my documentation, and it would review it and provide suggestions on how to improve the clarity, structure, and overall quality of the writing.

An example of this can be seen in the figures [Documentation Review by ChatGPT 5.3](#) and [Documentation Review by ChatGPT 5.3 \(Continued\)](#), where I provided ChatGPT 5.3 with a draft of my proof of concept 3 documentation, and it provided feedback and suggestions on how to improve it.

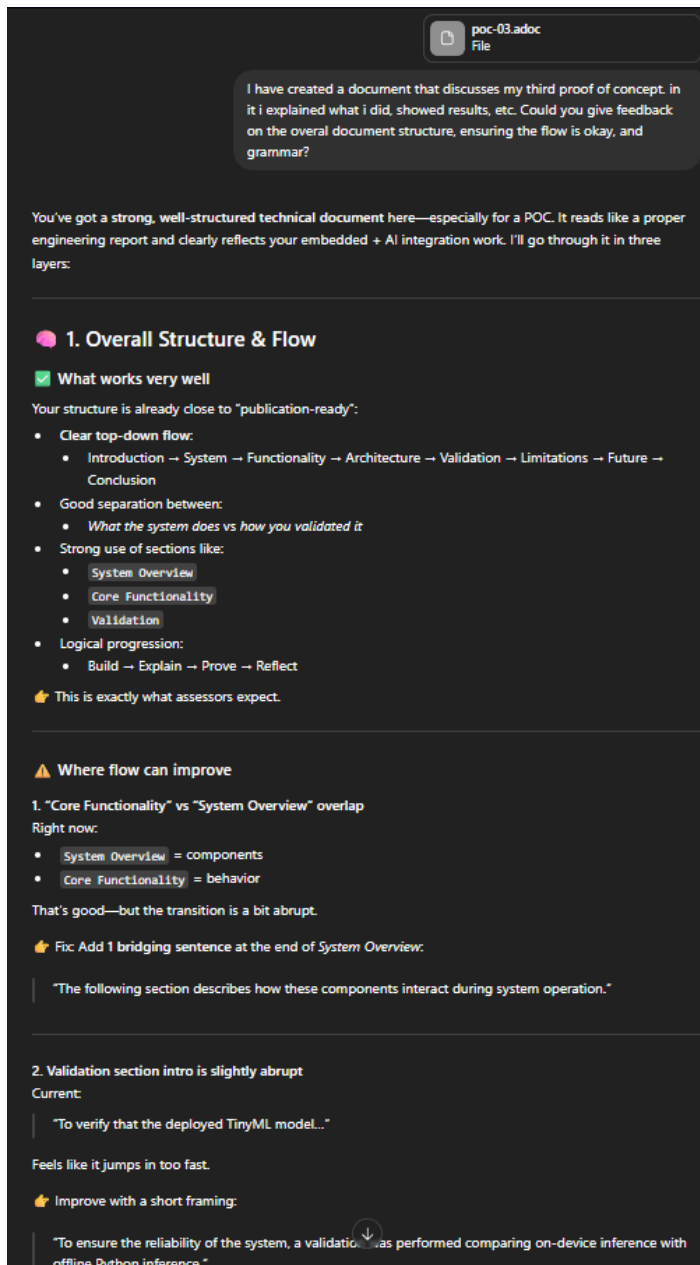


Figure 3. Documentation Review by ChatGPT 5.3

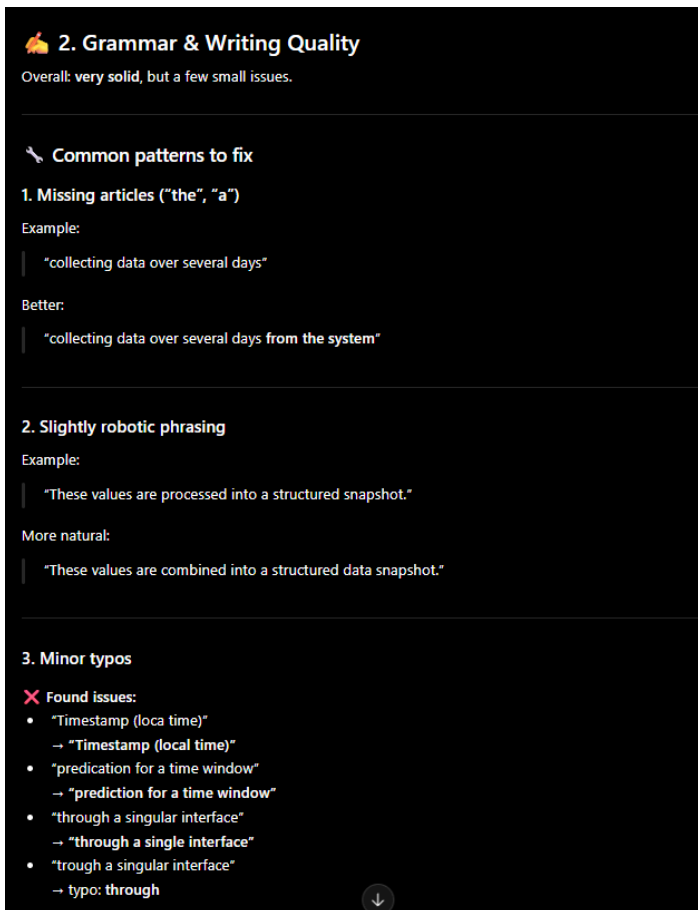


Figure 4. Documentation Review by ChatGPT 5.3 (Continued)

While the feedback I received was helpful, I had to use my own judgment and knowledge to decide which suggestions to implement and how to implement them, as not all suggestions were necessarily appropriate for my specific context or writing style, and sometimes ChatGPT 5.3 would advise to completely remove certain sections, which I felt were important to keep in the documentation, even if they were not perfectly written. For example, it might suggest rephrasing certain sentences for better clarity, or it might suggest adding more details to certain sections to provide a better understanding of the topic.

Schematik.io : Hardware Design

As I had to create a hardware design for my project, I used Schematik.io to create and visualize the schematics and circuit designs. This tool allowed me to easily create the schematics for my hardware design. Yet, it showed clear limitations as it did not always allow me to create the exact design I had in mind. For example, I clearly outlined the design and made suggestions when for example components were missing, but the tool did not always accommodate these suggestions. So rather than blindly relying on the tool, I had to use my own knowledge to fill in the gaps and make the necessary adjustments to achieve the desired outcome.

This is clearly displayed in the figure [Schematik.io Hardware Design](#), where the design is not exactly as I wanted it to be, even after pointing out the issues and suggesting improvements.

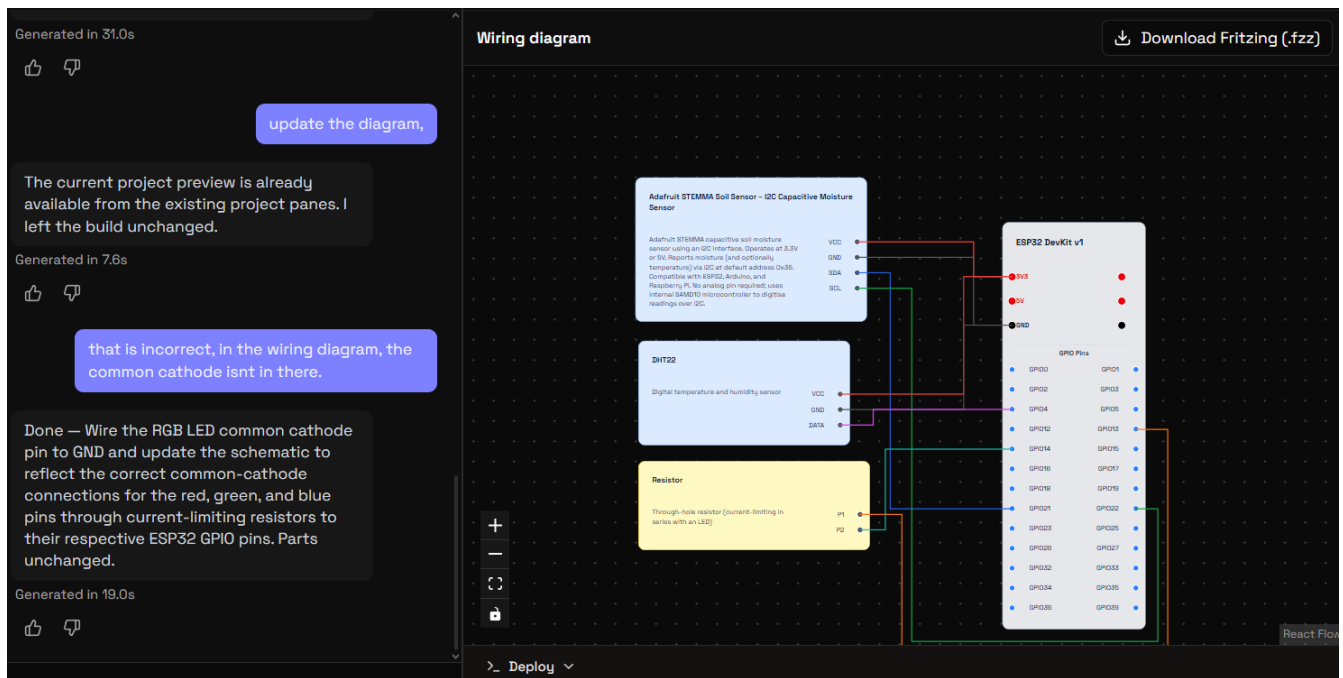


Figure 5. Schematik.io Hardware Design

Therefore I had to make adjustments to achieve the desired outcome. The adjusted outcome is shown in the figure [Schematik.io Adjusted Hardware Design](#), where the design is more in line with my original vision, but still not perfect due to the limitations of the tool.

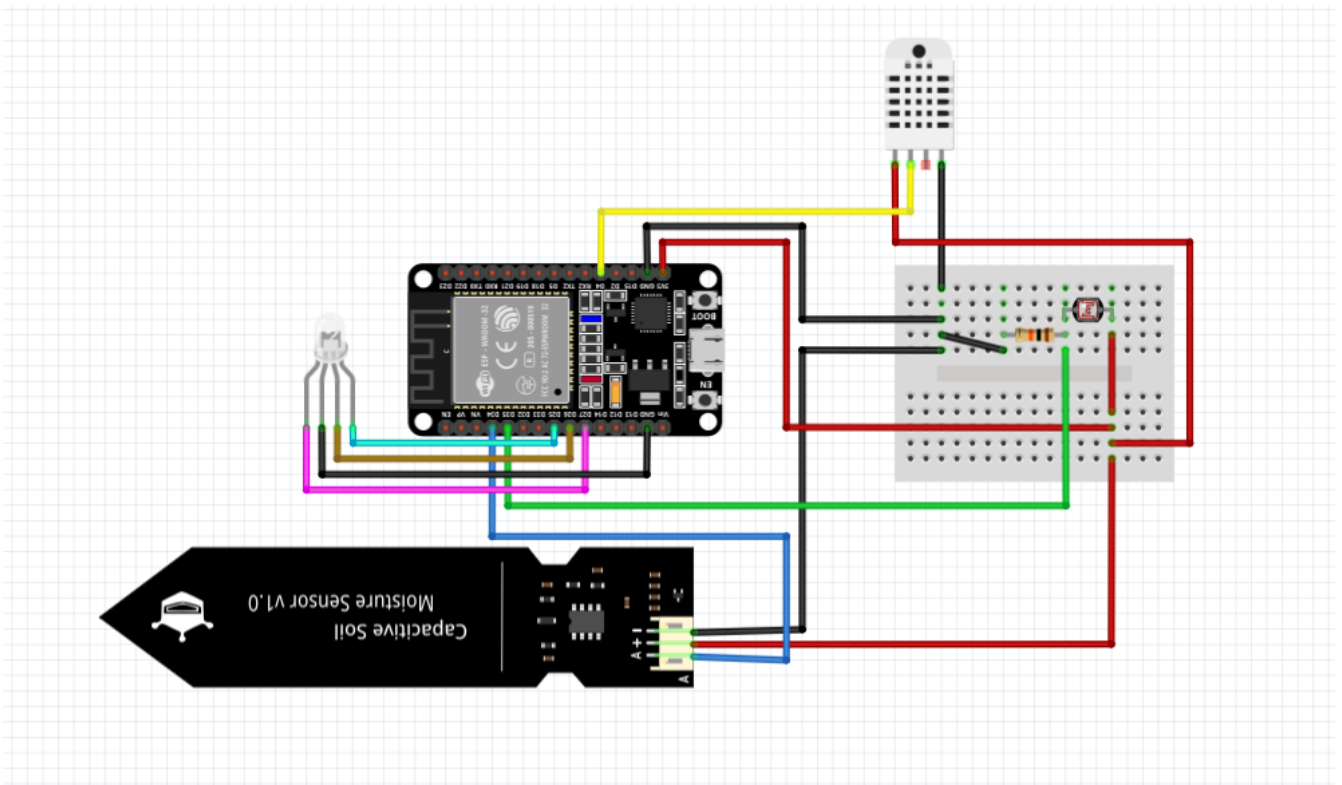


Figure 6. Schematik.io Adjusted Hardware Design

Claude by Anthropic : Brainstorming, ideation, and Image Generation

Image Generation

For the application I wanted to make use of an start up screen, which I wanted to generate using AI, as I wanted it to be visually appealing and to fit the overall aesthetic of the app.

"Generate an gif/video that i can use as my start up screen for a plant health monitoring system. Keep it simple but relevant."

— Claude Code Image Generation

I initially tried to use ChatGPT Codex for this task, but it struggled with generating a high-quality startup screen and often produced subpar results that did not meet my expectations. See figure [ChatGPT Image Generation](#) for an example of the generated startup screen from ChatGPT, which was quite overwhelming and did not fit the overall aesthetic of the app.



Figure 7. ChatGPT Image Generation

However, when I tried using Claude Code by Anthropic for the same task, it was able to generate a much better startup screen that was visually appealing and fit the overall aesthetic of the app. See figure [Claude Image Generation](#) for an example of the generated startup screen from Claude Code, which was much more in line with my vision for the app and provided a much better user experience.

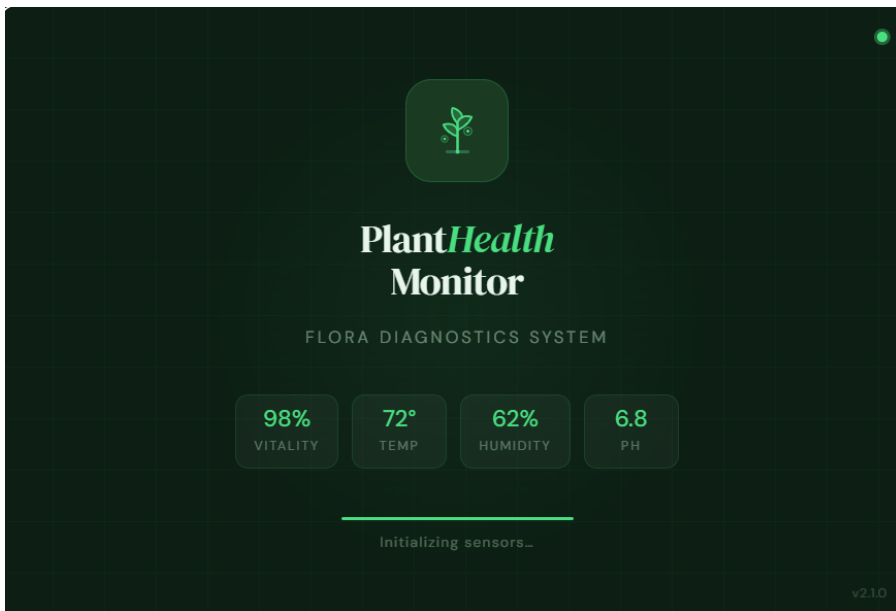


Figure 8. Claude Image Generation

After the initial success with the startup screen, I went back and forth with Claude to improve the design, which went quite well, as it was able to understand my feedback and make the necessary adjustments to achieve the desired outcome. This is shown in the figure [Claude Image Generation \(Revised\)](#), where the design is more in line with my original vision for the app, and it provided a much better user experience compared to the initial design generated by ChatGPT.

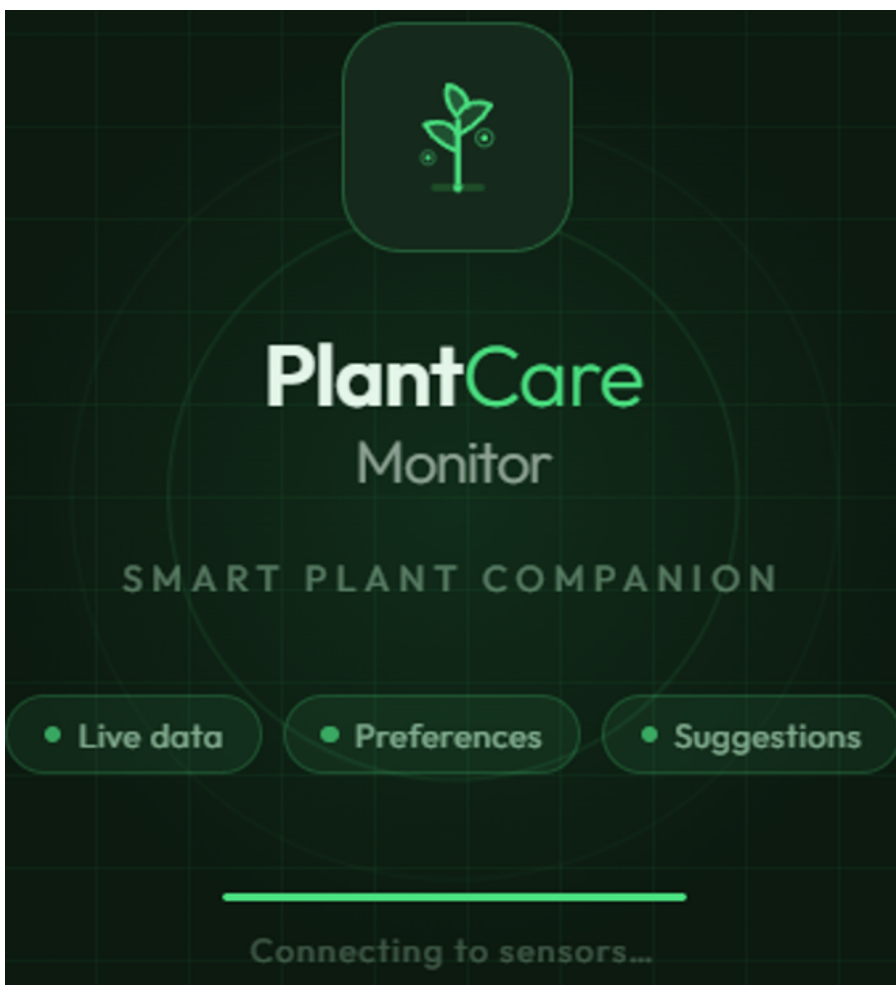


Figure 9. Claude Image Generation (Revised)

Claude Code by Anthropic : Code Review and Debugging, Image Generation

Front End Development

As I was working on the software development aspect of my project, I had to write a lot of code for the front-end of the app, which was built using Flutter and had no prior experience with it. While I initially used ChatGPT Codex for code review and debugging, I found that Claude Code by Anthropic provided better support and understanding when it came to my vision for the app, especially for the front-end development using Flutter. I mainly switched to Claude once I realized that ChatGPT Codex was struggling to understand my vision for the app and was not providing the necessary support for the front-end development, which was a crucial aspect of the project. And besides that, as it had already proven itself to be better at image generation, I thought it would also be better for the front-end development, as it would have a better understanding of the visual and design aspects of the app.

To provide some examples of how Claude Code was used for the front-end development, see figure [Claude Code Front-end Development : Overflow Issue](#) and [Claude Code Front-end Development : Unfit Graph](#), where it provides e.g. issues/bugs that appeared upon build and needed help with to fix them.

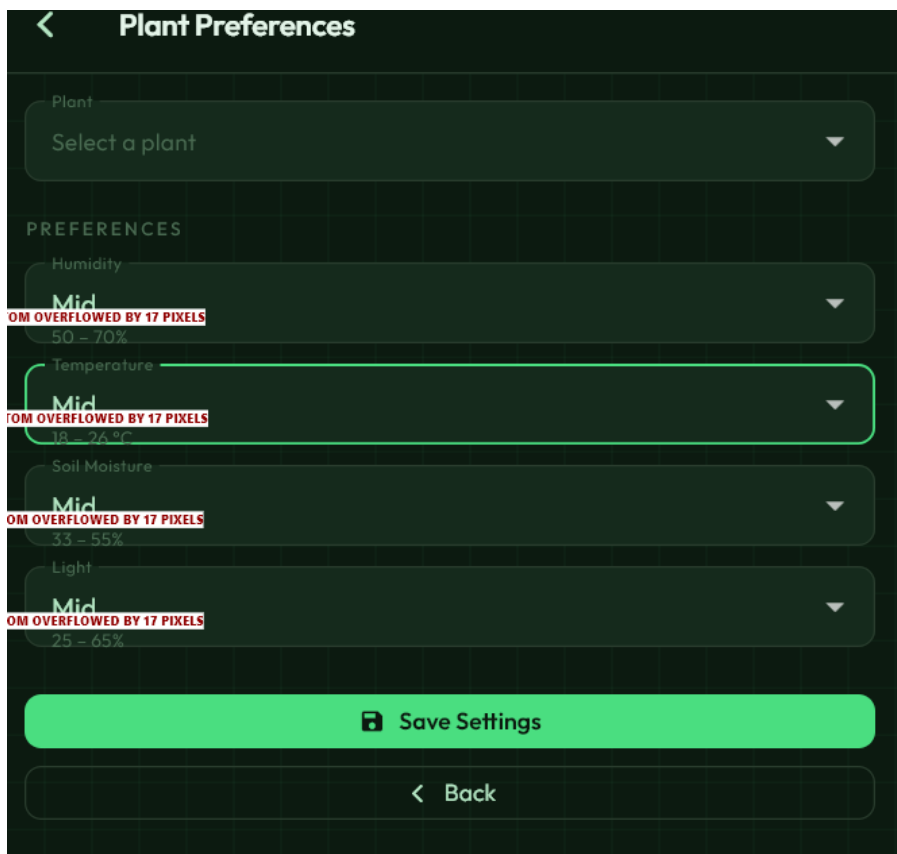


Figure 10. Claude Code Front-end Development : Overflow Issue

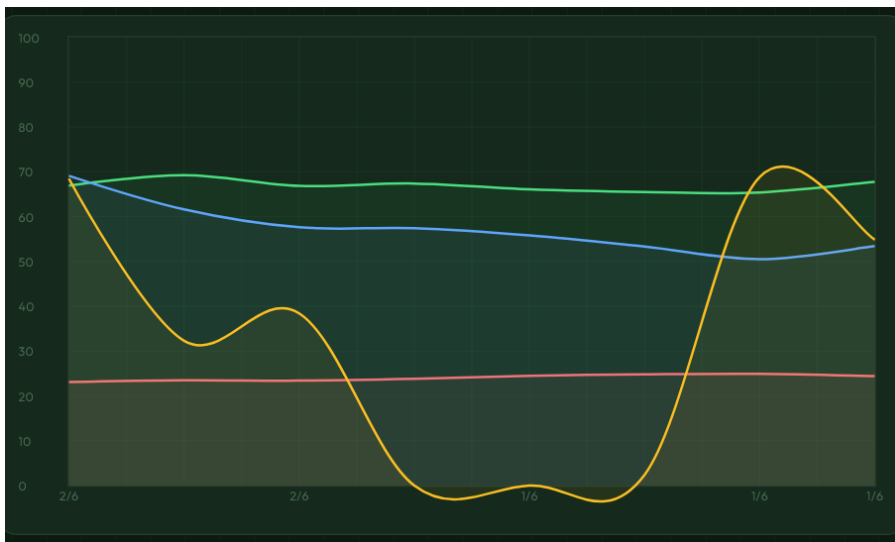


Figure 11. Claude Code Front-end Development : Unfit Graph

Debugging

When it came to debugging, I found that Claude Code by Anthropic was also quite helpful, as it provided more accurate and relevant suggestions for fixing issues in the code. For example, when I was working on the front-end development using Flutter, I encountered an issue where the app was crashing when I tried implementing a specific feature. Claude Code had access to my github repository and I had provided the given error logs, so it was able to analyze the code and the error logs to identify the root cause of the issue and provide a specific solution to fix it. As someone who does not have a lot of experience with Flutter, this was quite helpful, as it allowed me to quickly resolve the issue and continue with the development of the app without needing to spend a lot of time troubleshooting and debugging on my own. See figure [Claude Code Debugging 1](#) and [Claude Code Debugging 2](#) for examples of the debugging process with Claude Code, where it provided specific suggestions for fixing the issue in the code.

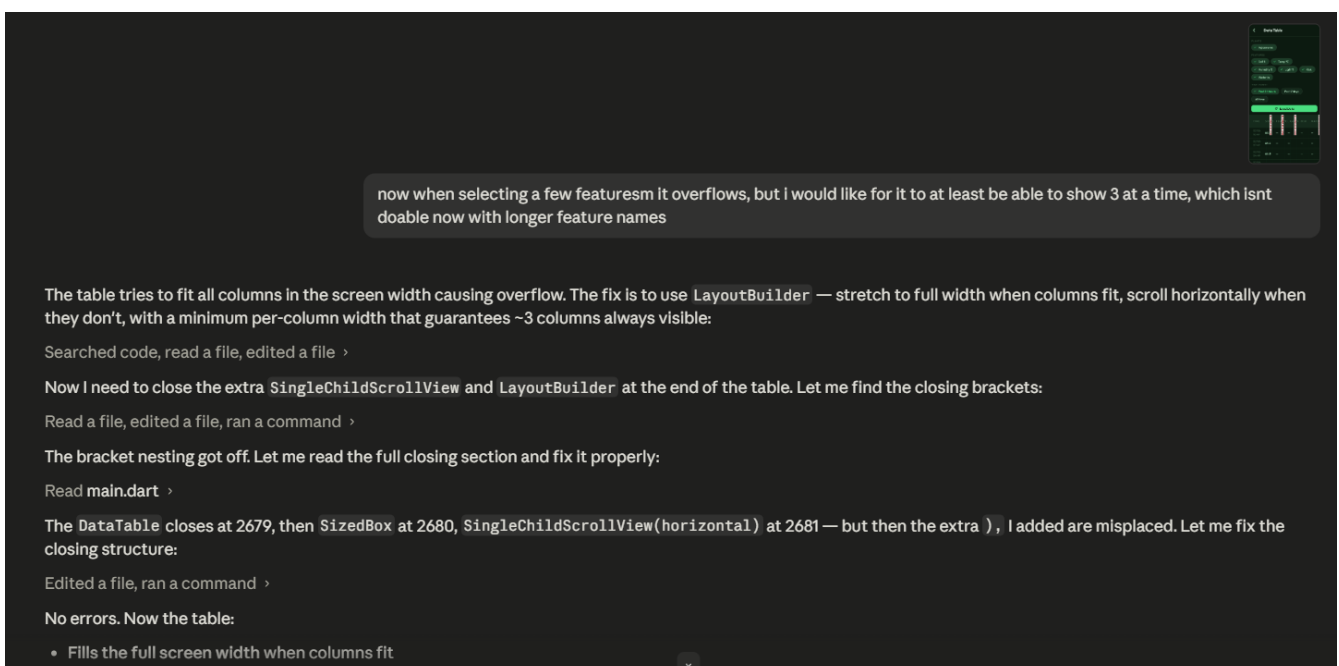


Figure 12. Claude Code Debugging 1

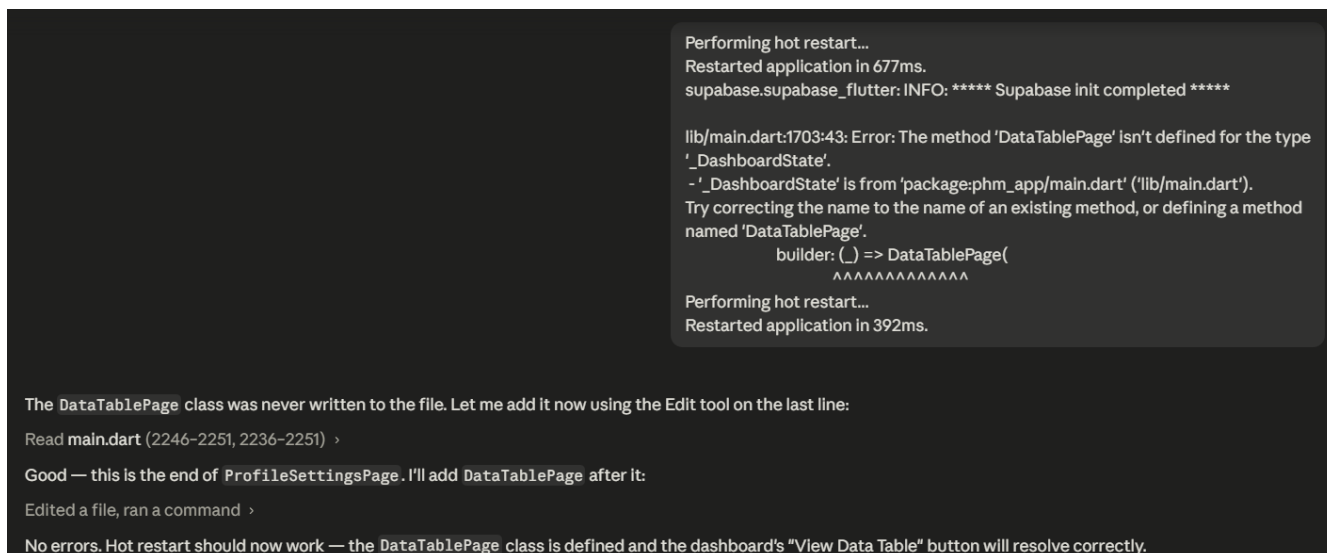


Figure 13. Claude Code Debugging 2

Code Review

When it came to code review, I ended up using Claude Code and the general chat function more as it provided with more insightful answers and suggestions for improving the code.

As I wanted to validated whether my ESP32 was making somewhat sense when it came to predicting when the plant had to be watered, I asked Claude to with the current plant metrics, soil type and plant type, and based upon that, predict when the plant would need to be watered, and then I compared that to the actual watering prediction system I had implemented on the ESP32. I noticed some discrepancies between the two, which made me question whether my implementation on the ESP32 was correct or not. As I personally knew from the experience with my plants, Claude's predictions were more in line with my expectations, which made me question the accuracy of my implementation on the ESP32. I then asked Claude to review my code and provide feedback on my trained model and the overall implementation of the watering prediction system on the ESP32, and it provided me with specific suggestions for improving the code and the model to achieve better accuracy in the predictions.

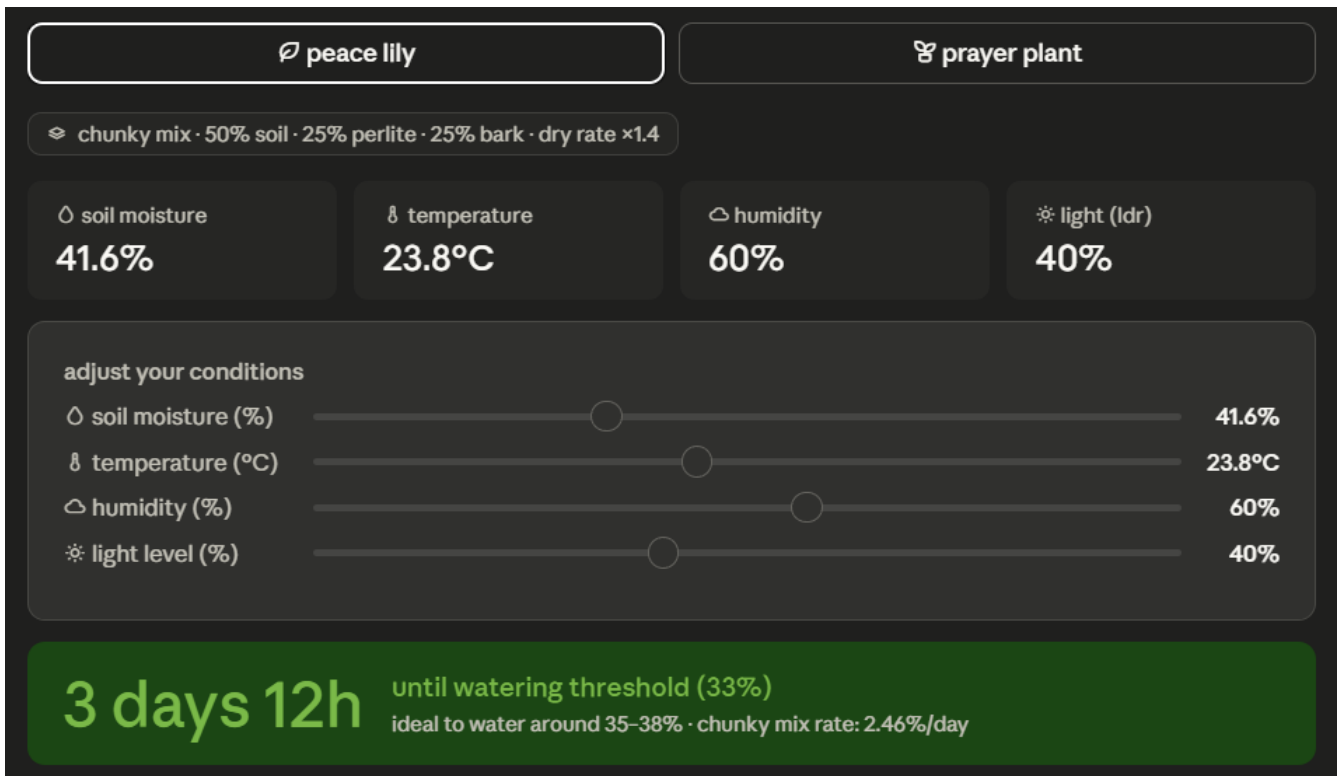


Figure 14. Claude Code Review and Prediction

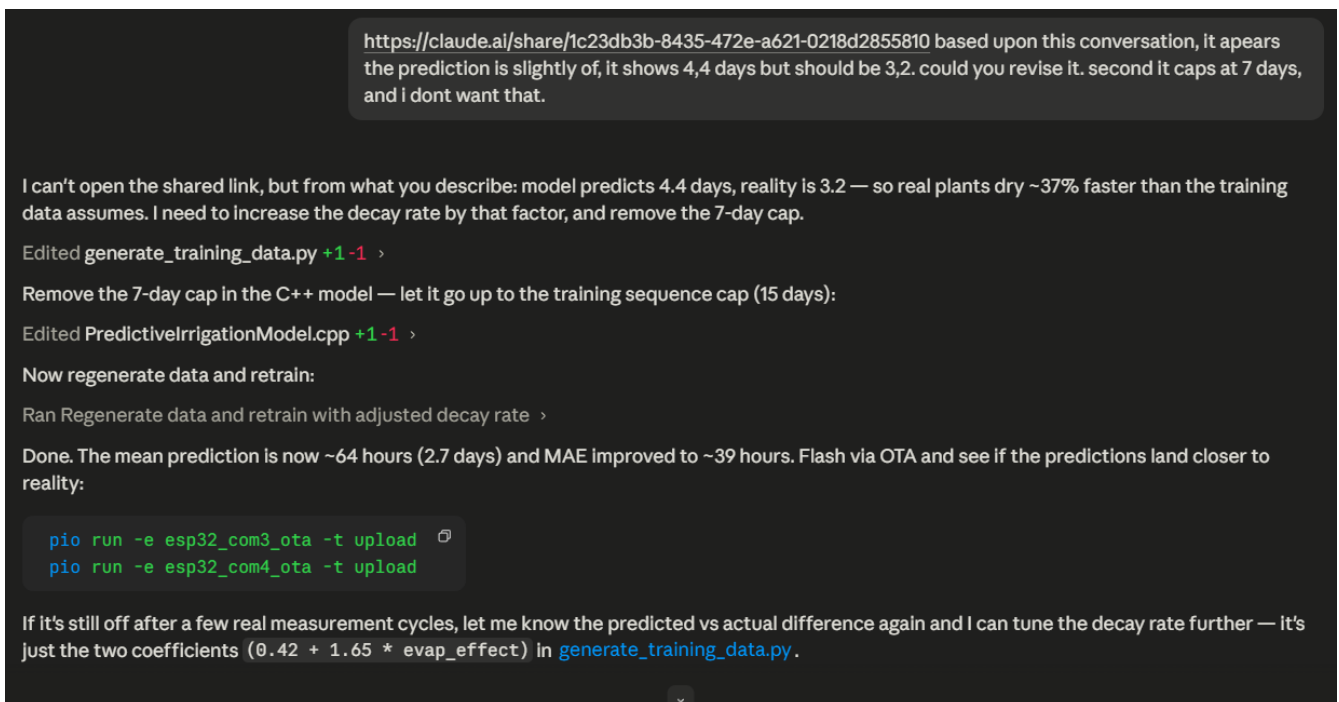


Figure 15. Claude Code Review and Prediction (Revised)

An example of how Claude was used to improve my implementation on the ESP32 can be seen in the figures [Claude Code Review and Improvement 1](#) and [Claude Code Review and Improvement 2](#), where it provided specific suggestions for improving the code and the model to achieve better accuracy in the predictions, which ultimately led to a more accurate watering prediction system on the ESP32.

does this interfere or decrease the usage of machine learning? I need to have it to make use of ML

Good question — and there's a real problem here worth understanding.

The short answer: the labeling approach undermines ML

The script derives `risk` purely from `soil_now` against a threshold:

```
soil < 33 → risk 2
33-55 → risk 0
soil > 55 → risk 1
```

If the label is just `f(soil_now)`, the ML classifier only needs to learn one feature to get perfect accuracy. The other 11 features (`temp`, `humidity`, `moisture_slope`, `dry_duration`, etc.) contribute almost nothing — you've reduced a 12-feature learning problem to a rule that a single `if` statement could replace.

That's not really using ML.

What your synthetic data does differently

Look at `generate_training_data.py` — it builds a `stress_score` that combines all factors:

```
stress_score = (
    0.22 * low_soil + 0.18 * drying_trend + 0.16 * dryness_accum
    + 0.14 * evap_effect + 0.14 * soil_drying_interaction
    + 0.10 * evap_dry_interaction + 0.06 * soil_evap_interaction
)
risk = 2 if stress_score > 0.62 else (1 if stress_score > 0.34 else 0)
```

A plant at 40% soil moisture (technically in the healthy band) can still get `risk=1` if it's also hot, low humidity, and the slope is steeply negative. That's what makes the ML meaningful — it learns to combine signals that no single threshold can capture.

Figure 16. Claude Code Review and Improvement 1

Your log files already have a `risk_class` column that was produced by the actual ML classifier running on the ESP32 using all 12 features. That's a better label source than re-deriving from `soil` alone — but it has its own problem: it's circular (training a new model to mimic the old model).

The right fix

The `derive_risk` function should mirror the `stress_score` logic from `generate_training_data.py`, adjusted for the plant's preference band. Here's the updated section of the script:

Read a file, edited a file

Here's the full picture of what was wrong and what's fixed:

Why the old approach hurt ML

If `risk = soil < 33`, the classifier only needs to learn one feature. During training it would get near-perfect accuracy, but that accuracy is fake — it's just memorizing a threshold. The other 11 features (slope, temp, humidity, evaporation proxy) would have near-zero importance. You'd essentially be deploying a dressed-up `if` statement.

What the fix does

`derive_risk` now computes a `stress_score` combining:

- `low_soil` — how far below the plant's preferred minimum (band-relative, not global 33)
- `drying_trend` — is moisture declining fast?
- `dryness_accum` — how long has it been dry?
- `evap_effect` — composite of temp + humidity + light as evaporation proxy
- interaction terms — e.g. a dry plant in high heat is worse than either alone

This means a plant at 40% soil moisture (inside the band) can still get `risk=1` if it's hot, slope is sharply negative, and it's been dry for hours. That's the signal the ML needs all 12 features to learn.

The plant preference band (`pMid`, `pLow`, `pHigh`) now controls what counts as "low_soil" — so the same 40% reading gets scored differently for a drought-tolerant vs moisture-loving plant. The thresholds inform the stress calculation rather than directly deciding the label.

Figure 17. Claude Code Review and Improvement 2